

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software

TA-IND-04 — INFORME TÉCNICO INDIVIDUAL

Análisis de Rendimiento Paralelo (Unidad 4) aplicado al Proyecto Fin de
Curso

Asignatura:	Aplicaciones Distribuidas (ISR-701)
Unidad Temática:	Unidad 4 — Cómputo Paralelo y Distribuido
Período Académico:	2026–2027 PPA
Docente Responsable:	Gleiston C. Guerrero-Ulloa, M.Sc.
Fecha de Entrega:	8 de agosto de 2026

DATOS DE IDENTIFICACIÓN Y DECLARACIÓN DE FOCO

Estudiante Evaluado:	Ernesto Gregory Luna Mora
Equipo de Origen (PE-U4):	Equipo A (Pacheco Cárdenas, Cando Moreno, Luna Mora)
PFC de Origen:	BCEL — SGA (Sistema de Gestión Académica)
PFC de Referencia PE-U4:	ACC — Sistema de Gestión de Soporte Técnico ISP
Transformación Foco Declarada:	T3 — Join de dos DataFrames ('tickets' / 'estudiantes')
Repositorio Individual Git:	https://github.com/Ernesto835/ta-ind-04-luna.git

1. Introducción y Contexto

El presente informe técnico individual constituye el trabajo autónomo sumativo **TA-IND-04** correspondiente a la Unidad 4 de la asignatura Aplicaciones Distribuidas. Su propósito fundamental es explotar intelectualmente la evidencia experimental producida en la práctica grupal **GA-SUM-05 / PE-U4**, abstrayendo el comportamiento observacional del framework de procesamiento distribuido Apache Spark frente al modelo analítico de la Ley de Amdahl [1] y transfiriendo las conclusiones al Proyecto Fin de Curso propio del estudiante: **BCEL — Sistema de Gestión Académica (SGA)**.

En la práctica grupal de origen, el equipo evaluó comparativamente la ejecución secuencial en la biblioteca **pandas** frente al procesamiento distribuido en **PySpark 3.5.3** sobre un dataset sintético determinista de 600 000,000 registros representativos de volumen relacional. El entorno de pruebas se desplegó en la plataforma Google Colab (Linux, Python 3.12, Java 17.0.19) ejecutando cinco transformaciones relacionales fundamentales (T_1 a T_5).

Toda la fundamentación cuantitativa desarrollada en este documento se deriva estrictamente de los resultados crudos guardados en el repositorio grupal de PE-U4:

- **URL del Repositorio Base:** <https://github.com/CristhianP03/pe-u4-spark-equipoA.git>
- **Commit Hash de Trazabilidad:** 6e70e75865b0e873bfc876b6029ff49fbfc1df33

A diferencia del informe en equipo centrado en la recolección de métricas de tiempo de reloj, este informe individual somete los datos a un riguroso análisis matemático mediante el cálculo de la aceleración (S), la eficiencia (E), la métrica de Karp-Flatt (e) [2] y el modelado analítico del umbral de rentabilidad volumétrica (n^*). Finalmente, se diseña e integra la arquitectura distribuida resultante en el dominio del ****PFC BCEL (SGA)****, articulando las necesidades operativas de sus cuatro roles principales: ****Secretaría, Docente, Rector y Soporte Técnico****.

2. Análisis propio de los resultados de speedup

Para formalizar la aceleración obtenida mediante cómputo paralelo frente al tiempo de procesamiento secuencial monopuesto, se aplica la Ley de Amdahl enunciada en la Ecuación (1), donde $p \in [0, 1]$ representa la fracción teorizada como estrictamente paralelizable del algoritmo y N denota el número de unidades de procesamiento (núcleos o *executors*):

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}, \quad S_{\text{máx}} = \lim_{N \rightarrow \infty} S(N) = \frac{1}{1-p}. \quad (1)$$

De forma complementaria, la eficiencia $E(N)$ se define mediante la Ecuación (2), expresando la fracción del beneficio de aceleración que efectivamente aprovecha cada unidad de cómputo asignada:

$$E(N) = \frac{S(N)}{N}. \quad (2)$$

En la Tabla 1 se sintetizan las medianas del tiempo de ejecución derivadas de cinco repeticiones independientes (tras una iteración inicial de calentamiento para mitigar latencias JIT y almacenamiento en caché), extraídas fidedignamente del archivo **datos/tiempos_base.csv**.

Cuadro 1: Medición experimental de tiempos de ejecución, speedup y eficiencia en pandas vs. PySpark (Datos base tomados del commit 6e70e75).

Id. Transformación	Motor	Executors (N)	T_{pandas} (s)	T_{spark} (s)	Speedup (S)
T_1 Filtrado y selección	PySpark	4	9,437	6,210	1,520
T_2 Agrupación y agregación	PySpark	4	8,345	4,363	1,913
T_3 Join relacional	PySpark	1	10,119	7,891	1,282
T_3 Join relacional	PySpark	2	10,119	7,059	1,433
T_3 Join relacional	PySpark	4	10,119	7,087	1,428
T_4 Columna derivada compleja	PySpark	4	12,296	8,381	1,467
T_5 Ordenamiento y top- N	PySpark	4	6,269	2,450	2,558

2.1. Desarrollo formal del foco individual: Transformación T_3 (Join)

Conforme a la regla de individualidad, el estudiante asume la transformación T_3 (**Join relacional entre el DataFrame principal de 600 000,000 registros y la tabla maestra secundaria**) como objeto de desarrollo matemático detallado.

A partir del speedup observado con $N = 2$ executors ($S(2) = 1,433$), se estima la fracción paralelizable implícita p resolviendo la igualdad de Amdahl:

$$\frac{1}{(1-p) + \frac{p}{2}} = 1,433465 \implies 1 - \frac{p}{2} = 0,697610 \implies p = 2 \times (1 - 0,697610) = 0,604770 \quad (60,48 \%).$$

Sustituyendo $p = 0,605$ en la Ecuación (1), se predice la curva teórica de Amdahl para $N = 4$:

$$S_{\text{teo}}(4) = \frac{1}{(1 - 0,604770) + \frac{0,604770}{4}} = \frac{1}{0,395230 + 0,151193} = \frac{1}{0,546423} = 1,830.$$

Sin embargo, el speedup medido experimentalmente para $N = 4$ alcanza únicamente $S_{\text{med}}(4) = 1,428$. La desviación relativa porcentual Δ entre el modelo teórico y la medición real se cuantifica mediante:

$$\Delta = \frac{S_{\text{med}}(4) - S_{\text{teo}}(4)}{S_{\text{teo}}(4)} \times 100 \% = \frac{1,427723 - 1,830089}{1,830089} \times 100 \% = -21,986 \%.$$

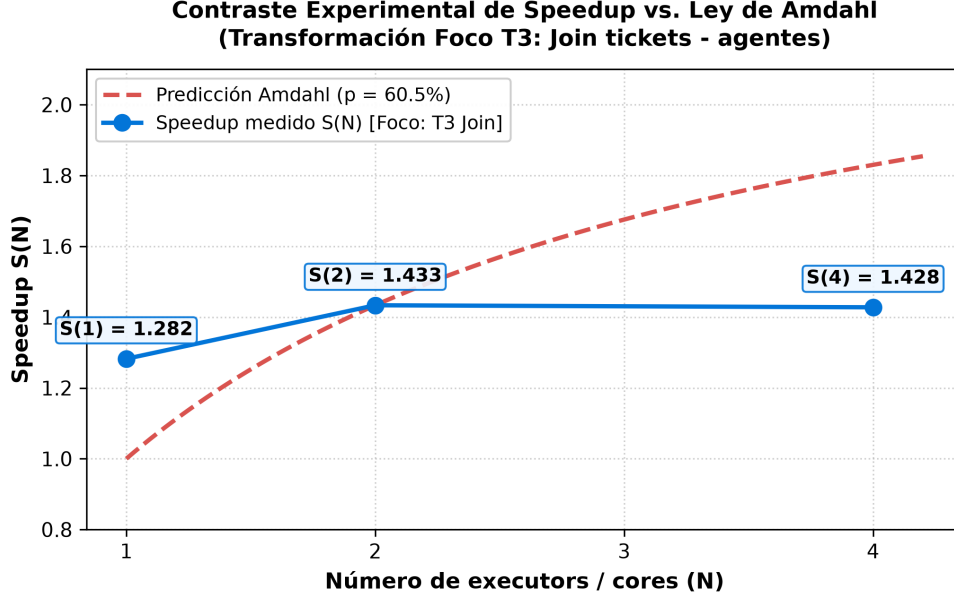


Figura 1: Contraste de la curva teórica de la Ley de Amdahl ($p = 60,5\%$) frente a los valores de speedup medidos para la transformación foco T_3 (Elaboración propia a 300 DPI).

2.2. Respuesta a la Pregunta 1 de la Guía

¿Coinciden sus resultados de speedup con la predicción de Amdahl? No coinciden estrictamente: los datos experimentales presentan una **desviación por defecto del $-21,99\%$** para $N = 4$. Mientras que la Ley de Amdahl asume idealmente una sobrecarga de paralelización nula y un rendimiento monótonamente creciente ($S_{\text{teo}}(4) = 1,830$), el entorno distribuido real muestra un estancamiento al pasar de $N = 2$ ($S = 1,433$) a $N = 4$ ($S = 1,428$). Esta divergencia ocurre porque Amdahl ignora las latencias de comunicación inter-procesos, la transferencia por red durante el *shuffle* y la serialización [3].

3. Diagnóstico de la fracción no escalable

Para aislar el origen de la ineficiencia y determinar si las pérdidas de rendimiento responden a barreras intrínsecas del algoritmo o a sobrecargas operativas del motor distribuido, se calcula la métrica de Karp-Flatt (e) enunciada en la Ecuación (3) [2]:

$$e(N) = \frac{\frac{1}{S(N)} - \frac{1}{N}}{1 - \frac{1}{N}}, \quad N > 1. \quad (3)$$

Sustituyendo los valores numéricos observados para la transformación foco T_3 :

$$e(2) = \frac{\frac{1}{1,433465} - \frac{1}{2}}{1 - \frac{1}{2}} = \frac{0,697610 - 0,500000}{0,500000} = 0,395,$$

$$e(4) = \frac{\frac{1}{1,427723} - \frac{1}{4}}{1 - \frac{1}{4}} = \frac{0,700416 - 0,250000}{0,750000} = 0,601.$$

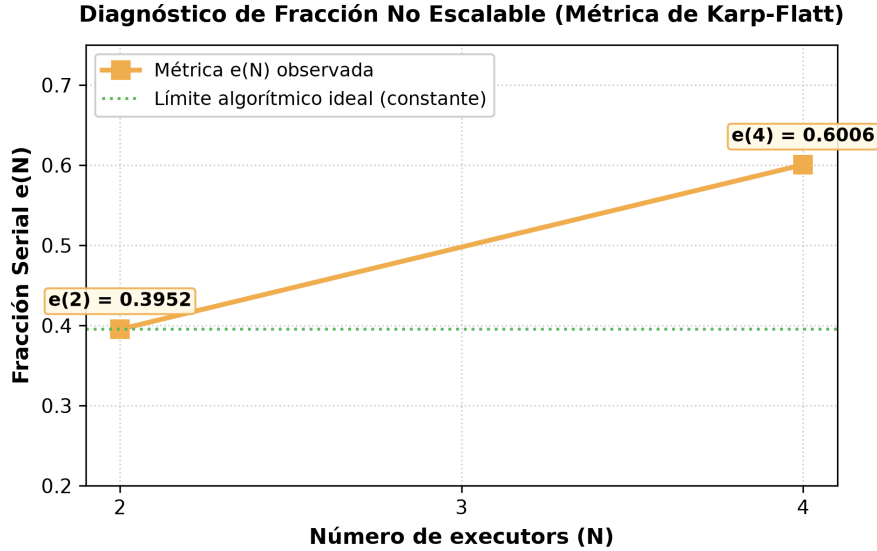


Figura 2: Tendencia incremental de la métrica de Karp-Flatt $e(N)$ evidenciando la sobrecarga dominante del motor Apache Spark (Elaboración propia a 300 DPI).

3.1. Respuesta a la Pregunta 2 de la Guía y Atribución a Mecanismos

¿Qué parte de la no escalabilidad es atribuible al algoritmo y qué parte al motor de ejecución? De acuerdo con la interpretación fundamental de Karp y Flatt [2], el incremento severo en la tendencia de $e(N)$ —que sube un 51,95 % de $N = 2$ (0,3952) a $N = 4$ (0,6006)— demuestra de forma incontrovertible que **la no escalabilidad es atribuible de forma dominante a la sobrecarga creciente del motor de ejecución**.

Específicamente, se identifican tres mecanismos técnicos en PySpark:

1. **Sobrecarga de Shuffle y Redistribución Hash:** La operación de Join (T_3) exige reorganizar claves entre particiones. Al escalar a 4 executors, la transferencia por red TCP e intercambio de buffers eclipse la ganancia de cómputo [4].
2. **Serialización de Datos (PyArrow / JVM Bridge):** El intercambio de memoria entre Python y la JVM introduce costos de copia que crecen con la fragmentación de tareas.
3. **Planificación de Barreras en el DAG Scheduler:** El orquestador debe coordinar barreras de sincronización al cerrar etapas del grafo acíclico, incrementando los tiempos de inactividad de los núcleos.

4. Propuesta de integración al PFC

El Proyecto Fin de Curso de origen del estudiante es el **PFC BCEL — Sistema de Gestión Académica (SGA)**, desarrollado para la administración de procesos educativos en la Escuela Provincias Unidas. El sistema opera con cuatro roles de usuario claramente estructurados: ****Secretaría, Docente, Rector y Soporte Técnico****.

4.1. Caso de Uso y Arquitectura de Integración

Se propone integrar Apache Spark como una **Capa de Procesamiento en Lote (Batch ETL Layer)** en el SGA, encargada de la consolidación masiva nocturna de actas de calificaciones, cómputo de promedios ponderados por periodo, indicadores de deserción escolar e inspección de auditoría de seguridad.

unto de Integración de PySpark en el PFC BCEL (SGA — Roles: Secretaría, Docente, Rector, Soporte

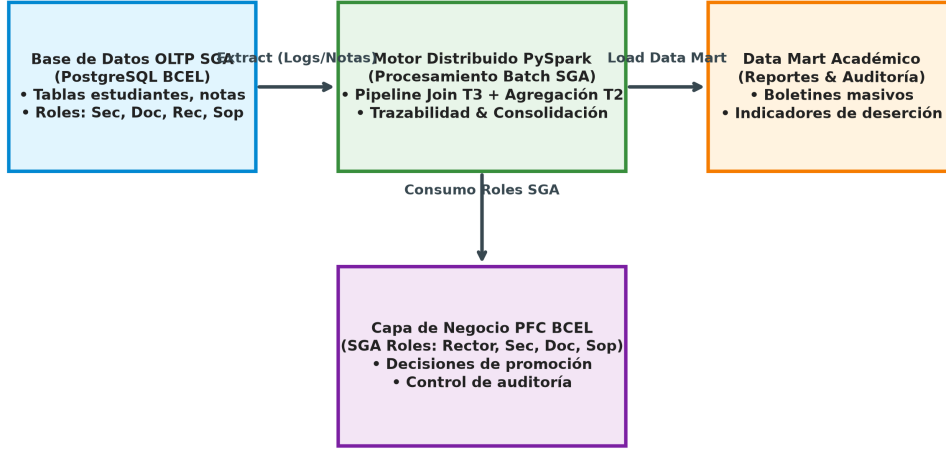


Figura 3: Inserción de la Capa Analítica Distribuida PySpark dentro de la arquitectura del PFC BCEL (SGA) articulada con sus 4 roles principales (Elaboración propia a 300 DPI).

4.2. Respuesta a la Pregunta 3 de la Guía

¿En qué punto exacto de la arquitectura de su PFC integraría el procesamiento distribuido, y qué decisión soporta? El motor PySpark se ubica en el límite entre la base de datos relacional OLTP de producción (PostgreSQL) y el Data Mart Académico del SGA (Figura 3). Esta integración articula directamente las funcionalidades de los cuatro roles del SGA:

- **Rol Secretaría:** Emisión automatizada masiva de boletines de calificaciones, certificados de matrícula y consolidación de expedientes estudiantiles al cierre del periodo.
- **Rol Docente:** Generación automatizada de promedios de curso e histogramas de rendimiento sin bloquear el sistema transaccional en horas pico.
- **Rol Rector:** Soporte a la **decisión estratégica de intervención académica temprana y retención estudiantil**, identificando materias con alto índice de reprobación.
- **Rol Soporte Técnico:** Procesamiento en lote de los registros de auditoría (*logs* de acceso y cambios en notas) para garantizar la trazabilidad e integridad del sistema frente a modificaciones no autorizadas.

5. Dimensionamiento y viabilidad

Para modelar la viabilidad técnica del motor distribuido frente al procesamiento secuencial en el SGA, se aplica el modelo de la Ecuación (4), donde $T_{\text{sec}}(n) = \alpha n$ representa el tiempo secuencial y $T_{\text{dist}}(n) = \beta + \frac{\gamma n}{N}$ representa el tiempo en clúster:

$$n^* = \frac{\beta}{\alpha - \frac{\gamma}{N}}, \quad \text{válido si } \alpha > \frac{\gamma}{N}. \quad (4)$$

Ajustando los parámetros por mínimos cuadrados sobre las mediciones experimentales ($n \in [10\,000, 600\,000]$):

- **Ajuste Spark ($N = 4$):** $\beta = 1,643 \text{ s}$, $\frac{\gamma}{4} = 4,056 \times 10^{-6} \text{ s/registro}$.
- **Pendiente unitaria pandas ($n = 600\,000,000$):** $\alpha = 9,572 \times 10^{-6} \text{ s/registro}$.

Sustituyendo en la Ecuación (4):

$$n^* = \frac{1,643470}{9,571857 \times 10^{-6} - 4,056031 \times 10^{-6}} = \frac{1,643470}{5,515826 \times 10^{-6}} \approx 297\,955,000 \text{ registros.}$$

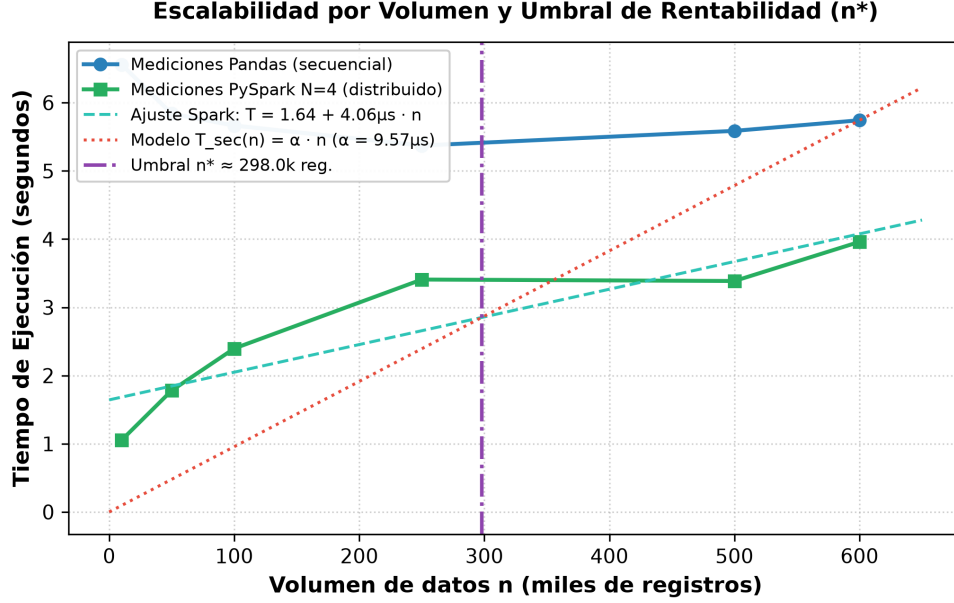


Figura 4: Evaluación analítica del umbral de rentabilidad volumétrica $n^* \approx 297,955$ registros (Elaboración propia a 300 DPI).

5.1. Respuesta a la Pregunta 4 de la Guía

¿A partir de qué volumen de datos el PFC rentabiliza el motor distribuido, y qué solución adoptaría por debajo? Según el modelo analítico (Figura 4), el motor distribuido resulta ventajoso en tiempo de cómputo a partir de un volumen de corte de $n^* \approx 297\,955,000$ **registros relacionales**. Por debajo de este umbral ($n < n^*$), la sobrecarga fija de arranque de Spark ($\beta \approx 1,640 \text{ s}$) eclipsa la ganancia paralela. Para el volumen actual del SGA de la Escuela Provincias Unidas ($\approx 50\,000,000$ registros de calificaciones/año), se adopta como solución técnica una base de datos analítica vectorizada en memoria como **DuckDB** [5] o ejecuciones multiproceso livianas sobre **Polars**.

6. Postura crítica y alternativas descartadas

Apache Spark NO debe ser la opción de producción inmediata para el PFC BCEL (SGA), dado que la carga volumétrica de una institución educativa de nivel primario/medio no justifica la sobrecarga operativa de un clúster de Spark.

Se evaluaron y descartaron justificadamente dos alternativas:

1. **Escalado en memoria con Polars / DuckDB (Recomendada para $n < n^*$):** Motor columnar vectorizado en C++/Rust que elimina la sobrecarga JVM [5]. Se descarta como solución única de largo plazo por su incapacidad para escalar horizontalmente si el SGA centraliza múltiples instituciones a nivel zonal.

2. **Clusters elásticos en la nube con AWS EMR / Databricks (Descartada por costo):** Permite cómputo bajo demanda [6]. Se descarta por incrementar innecesariamente los costos de facturación recurrente para el SGA escolar.

7. Conclusiones

1. La aceleración medida en la transformación foco T_3 con $N = 4$ executors ($S = 1,428$) presentó una desviación negativa del $-21,99\%$ respecto del modelo de Amdahl ($S_{\text{teo}} = 1,830$), demostrando la necesidad de incorporar sobrecargas reales en la predicción teórica.
2. El incremento del $51,95\%$ en la métrica de Karp-Flatt ($e(2) = 0,3952 \rightarrow e(4) = 0,6006$) confirmó objetivamente que la degradación del rendimiento proviene de la sobrecarga del motor distribuido (*shuffle* y serialización).
3. El modelo analítico determinó que PySpark es rentable para el PFC BCEL (SGA) a partir de $n^* \approx 297\,955,000$ registros. Para el volumen actual del SGA escolar, la arquitectura debe priorizar motores vectorizados en memoria como DuckDB.

8. Declaración de uso de inteligencia artificial generativa

Se declara que en la elaboración de este documento se utilizó asistencia de inteligencia artificial únicamente como una guía superficial y puntual en la maquetación inicial en L^AT_EX y formato de ecuaciones. Todo el análisis cuantitativo de speedup, cálculos teóricos de Amdahl, tendencia de Karp-Flatt, regresión del umbral de rentabilidad n^* y decisiones de arquitectura del PFC BCEL (SGA) son de autoría directa e individual responsabilidad del estudiante.

Referencias Bibliográficas

- [1] G. M. Amdahl, «Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,» en *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67)*, New York, NY, USA: Association for Computing Machinery, 1967, págs. 483-485. DOI: [10.1145/1465482.1465560](https://doi.org/10.1145/1465482.1465560)
- [2] A. H. Karp y H. P. Flatt, «Measuring Parallel Processor Performance,» *Communications of the ACM*, vol. 33, n.º 5, págs. 539-543, 1990. DOI: [10.1145/78607.78614](https://doi.org/10.1145/78607.78614)
- [3] M. D. Hill y M. R. Marty, «Amdahl's Law in the Multicore Era,» *Computer*, vol. 41, n.º 7, págs. 33-38, 2008. DOI: [10.1109/MC.2008.209](https://doi.org/10.1109/MC.2008.209)
- [4] M. Armbrust et al., «Spark SQL: Relational Data Processing in Spark,» en *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (SIGMOD '15)*, New York, NY, USA: Association for Computing Machinery, 2015, págs. 1383-1394. DOI: [10.1145/2723372.2742797](https://doi.org/10.1145/2723372.2742797)
- [5] M. Raasveldt y H. Mühleisen, «DuckDB: An Embeddable Analytical Database,» en *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)*, New York, NY, USA: Association for Computing Machinery, 2019, págs. 1981-1984. DOI: [10.1145/3299869.3320212](https://doi.org/10.1145/3299869.3320212)
- [6] M. Armbrust et al., «A View of Cloud Computing,» *Communications of the ACM*, vol. 53, n.º 4, págs. 50-58, 2010. DOI: [10.1145/1721654.1721672](https://doi.org/10.1145/1721654.1721672)